Introduction to Computer Science

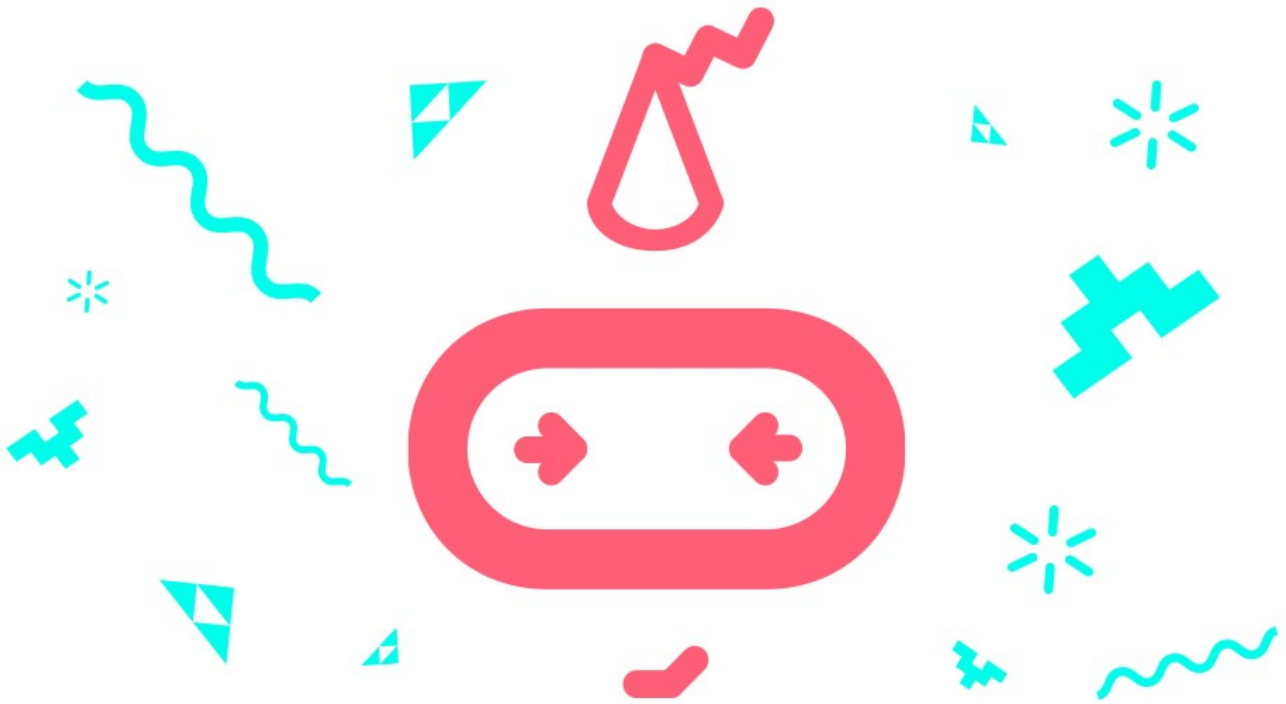
Unit 9:

Bits, bytes, and binary

Student workbook

makecode.microbit.org

Table of Contents



Overview

Unit summary

This unit introduces the concept of binary digits and a binary, or base-2, numbering system. In the unplugged activity, you will learn about bits and bytes. You'll use a binary vending machine to explore differences in the decimal (base-10) and binary (base-2) numbering systems by converting numbers between both systems. In the coding activity, you will learn how data is stored digitally and how it can be read and accessed in a program by coding the micro:bit to convert binary and decimal numerals. In the project, you will invent a paper and cardboard version of a binary counter, then program it to display the decimal value of binary numbers.

Learning goals

During this unit, you will:

- Understand what bits and bytes are and how they relate to computers and the way information is processed and stored.
- Learn to count in binary (base-2) and translate numbers from decimal (base-10) to binary and decimal.
- Apply the above knowledge and skills to create a unique program that uses binary counting as an integral part of the program.

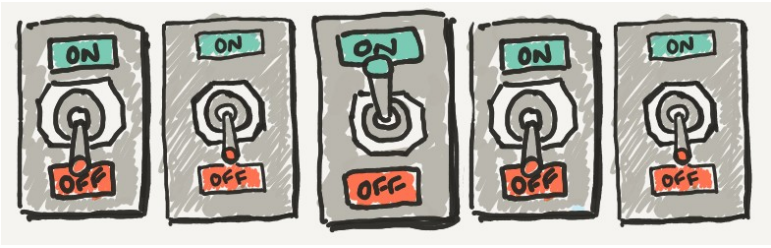
Lesson A: Understanding bits, bytes, and binary

Use this space to write your questions and ideas

Overview: Bits, bytes, and binary

Most everyone who uses a computer has heard the terms, Kilobyte (KB), Megabyte (MB), Gigabyte (GB) and even Terabyte (TB), usually when referring to the size of computer files and hard drives as well as download speeds. Bandwidth or connection rates are measured in bits/second. But what is a bit and what is a byte, and what do they have to do with computers?

Picture a basic room light. The light is either on or off. You control the current state of the light by flipping a switch that has only two settings, on or off. The earliest computers used a series of mechanical switches to control the flow of electricity through their circuits, turning each one on or off. The on/off states of the circuits were used to represent and even store information. The smallest unit of information, representing the state of one switch, is known as a bit.



A bit is a binary digit and has only two possible values, zero or one. The value of the bit represents the current state of a single switch. If the switch is off, then the bit has the value zero. If the switch is on, then the bit has the value one.

A bit can only represent two different values, 0 or 1. To represent larger pieces of information, bits are strung together in sequences of eight called bytes.

A byte is a sequence of binary digits made up of eight bits.

A byte can represent any value from 00000000 through 11111111, for a total of 256 different possible values. Each digit in a byte can be thought of as representing an individual switch that is either off (0 or on (1).

Modern computers rely on transistors, which pack millions of tiny switches into a chip smaller than your thumb, but information is still represented in essentially the same way: as a series of 1s and 0s. By using binary, computers can represent information simply and efficiently using a system that is very effectively modeled in digital circuitry.

In coding:

- The 1s and 0s of bits and bytes can be used to represent letters, numbers, and even different keys on a computer keyboard.
- A bit can be used to hold a Boolean (true/false) value. A value of 0 represents “false” and a value of 1 represents “true.”

List of definitions

Term	Definition

bit	A binary digit with two possible values: 0 or 1
byte	A sequence of eight bits and has 256 possible values from 00000000 through 11111111
Kilobyte (KB)	1,024 bytes or 2^{10} bytes
Megabyte (MB)	1,048,576 bytes or 2^{20} bytes
Gigabyte (GB)	1,073,741,824 bytes or 2^{30} bytes
Terabyte (TB)	1,099,511,627,776 bytes or 2^{40} bytes

Binary vending machine

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

Use this space to write your questions and ideas

Lesson B: Code a binary transmogrifier

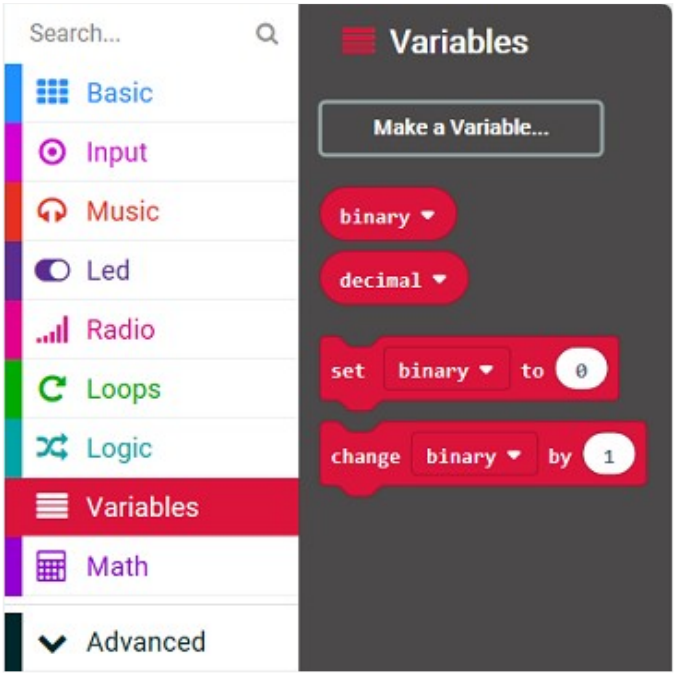
Coding activity: Binary calculator

You will be building a binary calculator with the micro:bit. The user will be able to use the buttons to enter binary 0s and 1s and will be able to press A+B at any time to display the decimal equivalent of the number that has been entered.

Create the Variables

You will need to create a number variable to hold the running decimal total. You should also create a string variable to hold the current binary number.

1. In Microsoft MakeCode, start a new project and name it something like binary calculator or binary converter. Either delete the ‘forever’ block in the coding Workspace or move it to the side as it’s not used in the activity. Then, from the Variables menu, select the Make a Variable button to make two variables. Name one decimal and the other: binary.



Initialize the Variables

When the program starts up, you should initialize your variables to starting values.

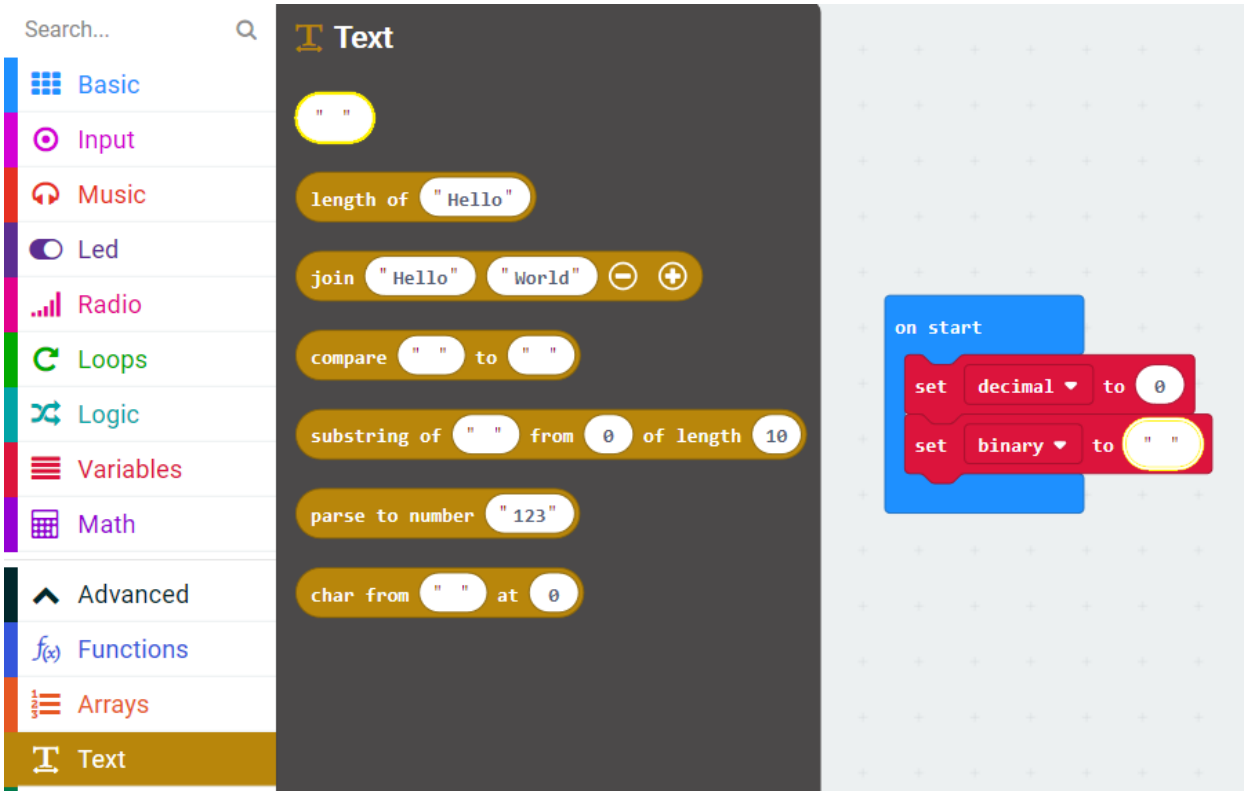
2. From the Variable Toolbox drawer, drag two ‘set’ blocks to the coding Workspace and drop them inside the ‘on start’ block. Depending on which variable you made first, the ‘set’ block will default to one of the new variables. Use the dropdown menu to set one block to binary and the other to decimal.

Now, you can give these variables some starting values:

- For the decimal variable, keep the default parameter of 0.

- For the binary variable, you want an empty string to hold the binary text value.

3. Select the Advanced tab in the Toolbox to open up more Toolbox categories. Then select the Text Toolbox drawer to find the empty string oval block. Drag one onto the Workspace and drop it into the ‘set binary to’ block replacing the 0.



By setting the binary variable to an initial value of “ “ you tell the micro:bit that it is a string variable: a literal string of characters. This is important because you will be adding to this string character by character.

Ready, set, calculate!

You are ready to start entering numbers. Remember that binary numbers are calculated based on the number of place values (“bits”), and as you enter 1s and 0s, the value changes. One way to calculate the decimal value is to wait until the user presses A+B, and then calculate the entire number based on the value of the string.

However, a much simpler method is to calculate the decimal number “on the fly”, which is to say, every time the user presses a 1 or a 0, calculate the current decimal value of that string so you only have to deal with one 0 or 1 at a time.

What's the pattern?

This is a table of the first fifteen binary numbers and their decimal equivalents. Your goal is to use this table to figure out how to calculate a new correct decimal value based on whether a user enters a 0, or a 1 as the next number in the string.

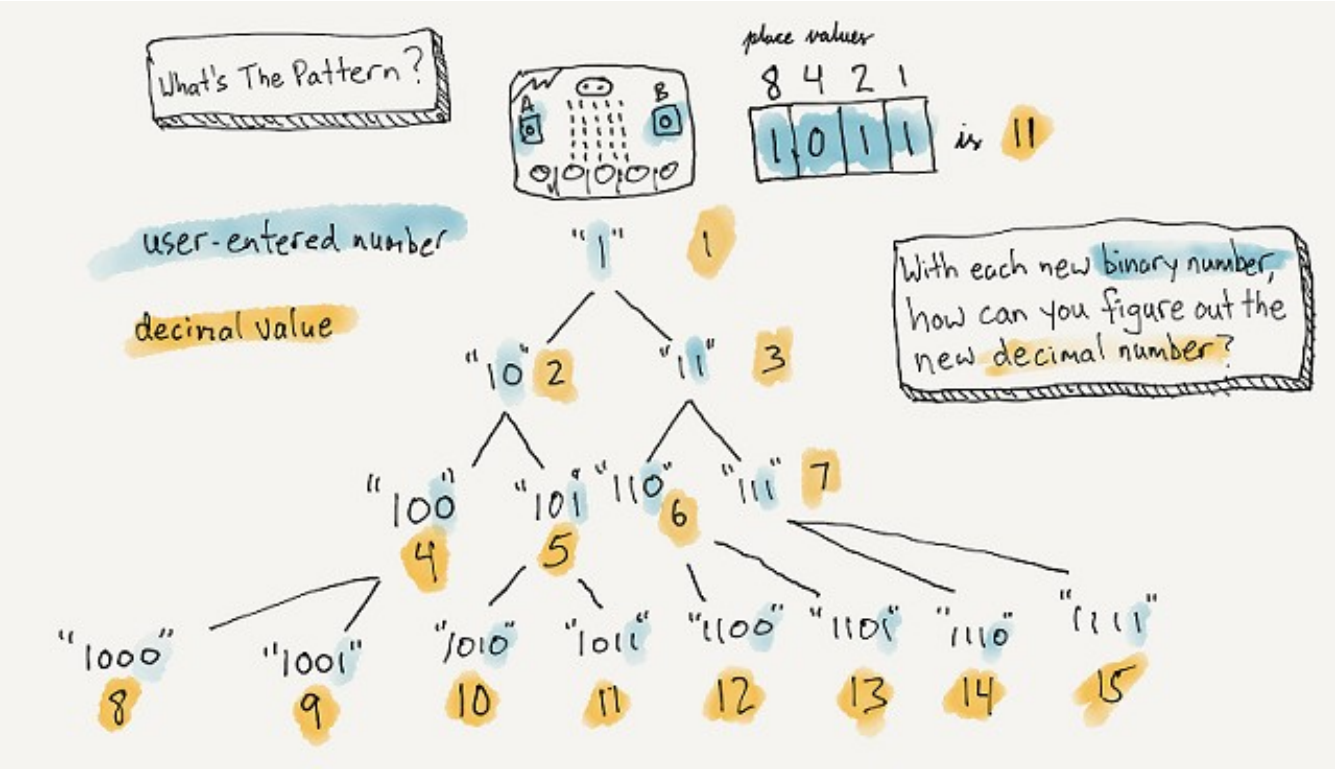
Binary	Decimal
1	1
10	2
11	3
100	4
101	5
110	6
111	7
1000	8
1001	9
1010	10
1011	11
1100	12
1101	13
1110	14
1111	15

For example, imagine you are the micro:bit. If the first number the human enters is a 1, you automatically know the new decimal value is a 1. If the second number that is entered is a 0, then your decimal value goes from 1 to 2. However, if the second number is also a 1, then your new decimal value goes from 1 to 3.

At that point, you either have a 10 or an 11 in your binary string. Let's take 10 as an example. The decimal value of binary 10 is 2. If the third number entered is a 0, then your new decimal value goes from 2 to 4. If the third number entered is a 1, then your new decimal value goes from 2 to 5.

If, on the other hand, you have 11 in your binary string, then your decimal value is 3. If the third number entered is a 0, then your new decimal value goes from 3 to 6. If the third number entered is a 1, then your new decimal value goes from 3 to 7.

See if you can spot a pattern that will help you figure out, for any given decimal value, what the new decimal value should be if the user enters a 0, or if the user enters a 1.



Pseudocode

The input for this program will be the buttons. Try to write out what should happen when each of the buttons is pressed. Here is one possible solution. Your own pseudocode might be different and that's okay. Remember there can always be different approaches in coding.

When Button A is pressed:

- Add a "1" to the end of the binary string.
- Show the current value of the binary string.
- Update the decimal value with the total.

When Button B is pressed:

- Add a "0" to the end of the binary string.
- Show the current value of the binary string.
- Update the decimal value with the total.

When Buttons A+B are pressed:

- Show the current value of the decimal string.

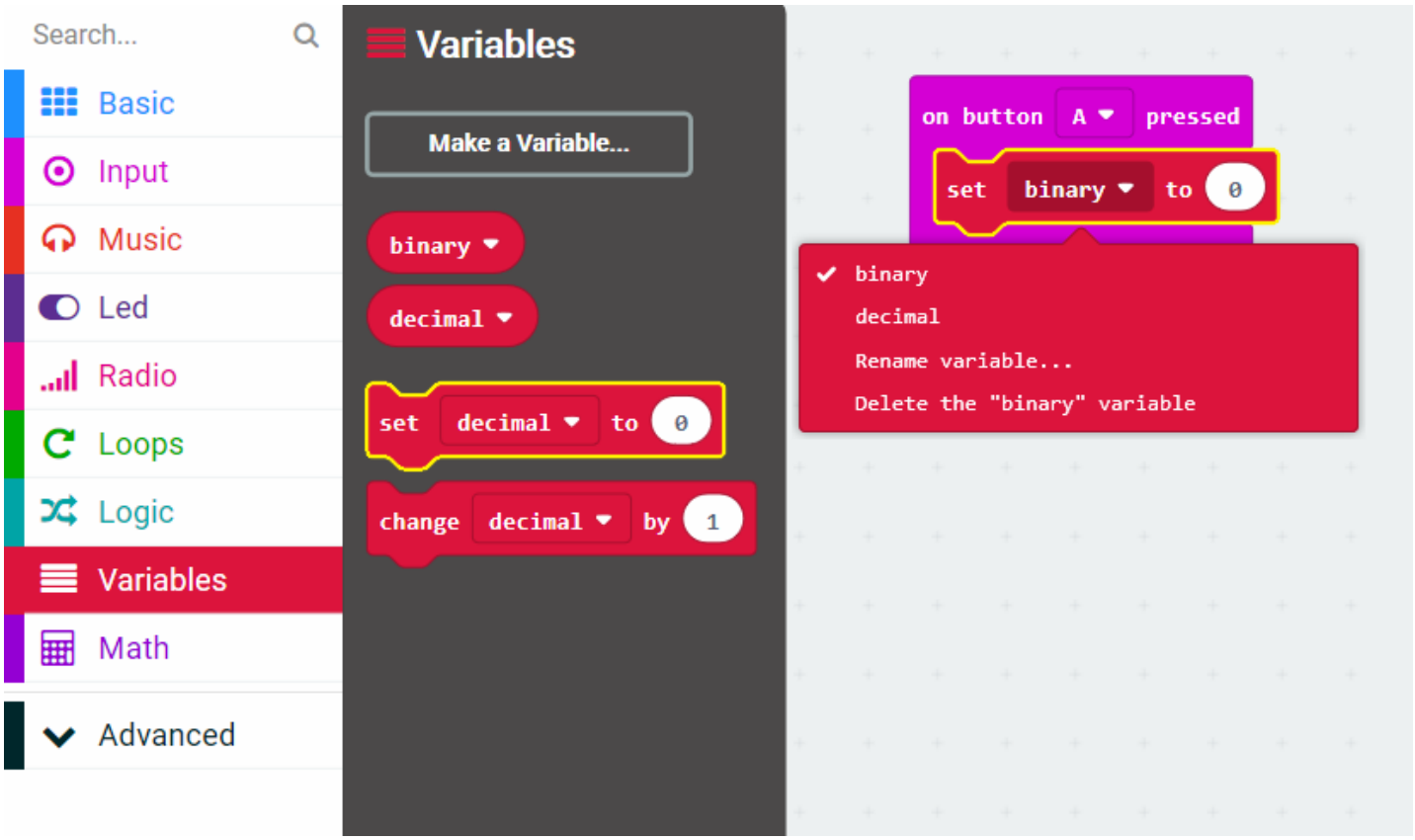
Use this space to write your pseudocode

Code button A

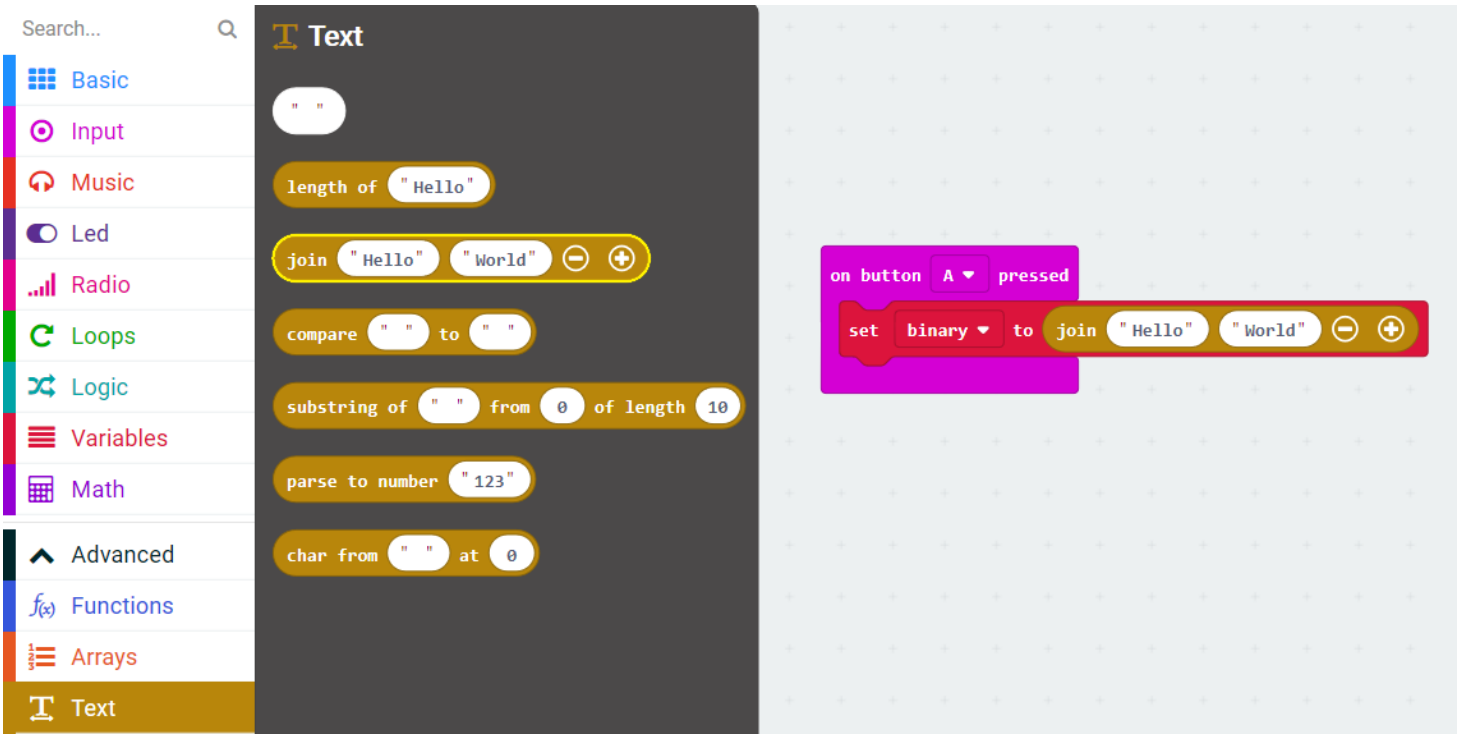
4. From the Input Toolbox drawer, drag three of the ‘on button A pressed’ event handlers to your coding Workspace. Leave one block with button ‘A’. Use the dropdown menus in the other two blocks to choose button ‘B’ and button ‘A+B’.

Now, work on what to do when button A is pressed. Button A represents a binary “1”. Your first task is to join a “1” to the existing string variable called binary.

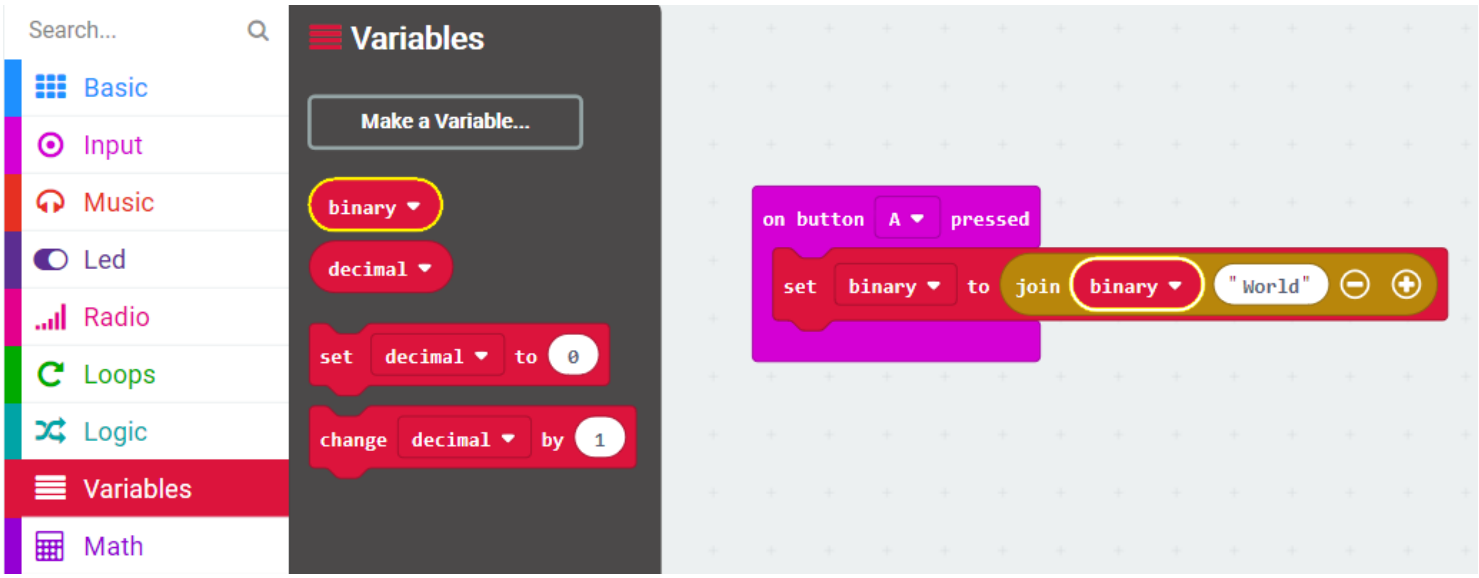
5. From the Variables Toolbox drawer, drag a ‘set (variable)’ block onto the Workspace, and drop it into the ‘on button A pressed’ block. Then, use the dropdown menu to select the ‘binary’ variable.



6. From the Text Toolbox drawer (under the Advanced menu), drag the ‘join’ block to your coding Workspace and drop it into the ‘set binary to’ block replacing the default 0 value.



7. From the Variables Toolbox drawer, drag a ‘binary’ variable value block onto the Workspace, and drop it into the first slot of the ‘join’ block, replacing the default “Hello”.

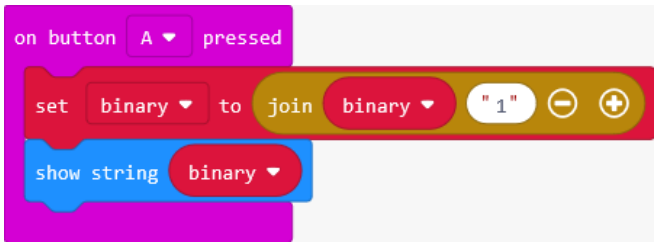


8. In the second slot of the ‘join’ block, replace the default value of “World” to 1. This will append a “1” to whatever value is currently being held in the binary variable.

Code button A display

9. Now, display this value on the micro:bit. From the Basic Toolbox drawer, drag a ‘show string’ block onto the Workspace, and drop it after the ‘set binary’ block in the ‘on button A pressed’ block.

10. From the Variables Toolbox drawer, drag a ‘binary’ variable value block onto the Workspace, and drop it into the ‘show string’ block replacing the default value of “Hello!”



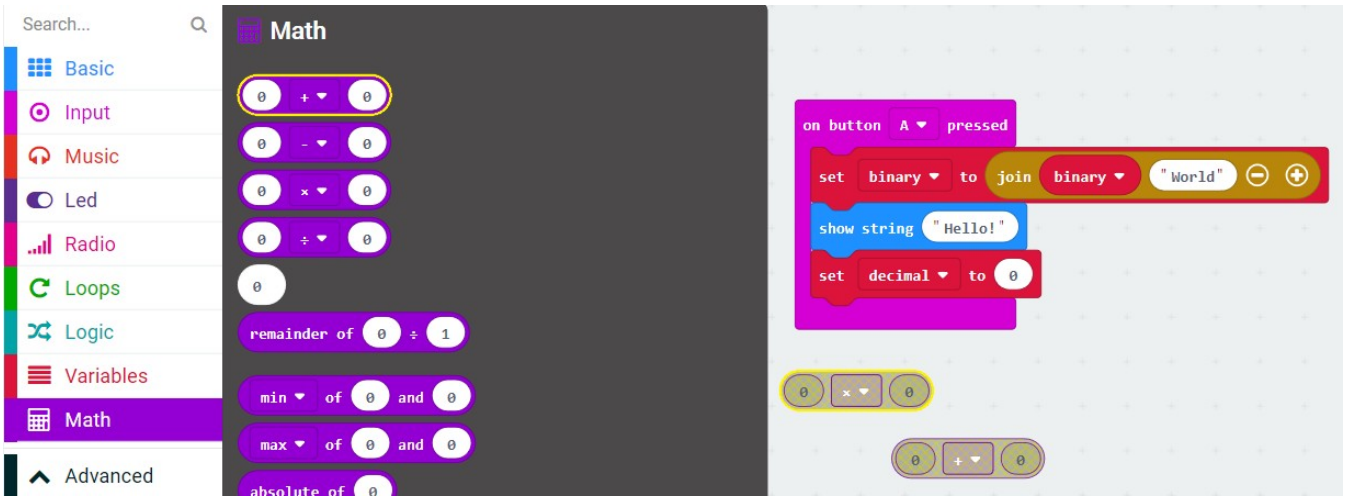
Finally, you will need to update the current decimal value with the value of the entire binary string. The pattern pseudocode is as follows:

- Whenever someone enters a 0, the new decimal value is twice the previous value.
- If someone enters a 1, the new decimal value is twice the previous value, plus 1.

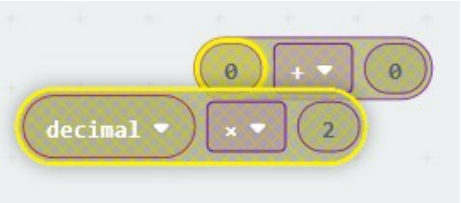
For button A (when a user enters a 1), you will need to use the ‘multiplication’ Math block and your binary variable block to create the proper formula. You will need to put that formula inside another Math ‘addition’ block in order to add one to the result.

11. From the Variables Toolbox drawer, drag a ‘set (variable)’ block onto the coding Workspace, and drop it into your ‘on button A pressed’ block after the ‘show string’ block. Then, use the dropdown menu to select the ‘decimal’ variable.
12. From the Math Toolbox drawer, drag out two blocks to the Workspace: the multiplication block and the addition block. Don’t place these yet; just keep them on the Workspace (they should be greyed out). You’ll be nesting the two math expressions.

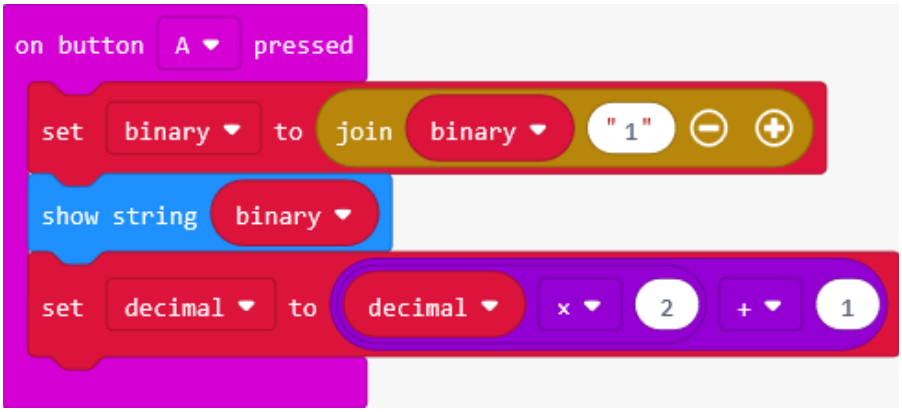
    Note: When working the nested expressions—Math or Logic blocks placed inside each other—it’s easier to do the block manipulation on the coding Workspace separately first before attaching to other blocks in your program.



13. From the Variables Toolbox drawer, drag a ‘decimal’ variable value block onto the Workspace and drop it into the first slot of the multiplication math block replacing the default value of 0. Type a 2 into the second slot of the multiplication math block. Now, drag the multiplication math block into the first slot of the addition math block, replacing the default value of 0.



14. Type a 1 into the second slot of the addition math block. Now, drag the whole Math expression into the ‘set (decimal) to 0’ block to replace the 0 default value.

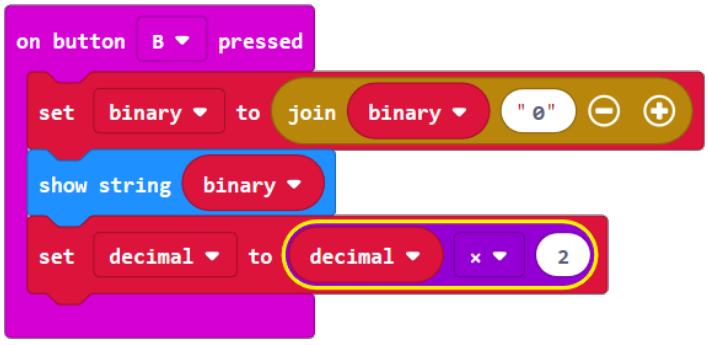


15. Test this code in the Simulator to confirm it’s working as intended.

Code button B

Your button B algorithm is similar, although you will be joining a “0” to the binary variable, and you are just multiplying the decimal variable by 2.

16. An alternative to coding the blocks from the Toolbox drawers is to duplicate the entire ‘on button A pressed’ set of blocks, then:
    - In the ‘join binary 1’ block, change the 1 to 0.
    - In the ‘set decimal to’ block, select the ‘decimal x 2’ oval and pull it out of the blocks to “un-nest” it (it will be grayed out). Delete the ‘0 + 1’ oval in the ‘set decimal to’ block and replace the resulting 0 value with the grayed out ‘decimal x 2’ oval block.



17. Again, test this new code in the Simulator to make sure it’s working as intended.

Code button A+B

Your button A+B algorithm just uses a ‘show’ block to show the current value of the decimal variable.

18. From the Input Toolbox drawer, drag an ‘on button A pressed’ block to the coding Workspace and use the dropdown menu to select ‘A+B’. Then, from the Basic Toolbox drawer, drag a ‘show number’ block and drop it into the ‘on button A+B pressed’ block. Then, duplicate a ‘decimal’ variable value from one of the other blocks and replace the 0 of the ‘show number’ block.

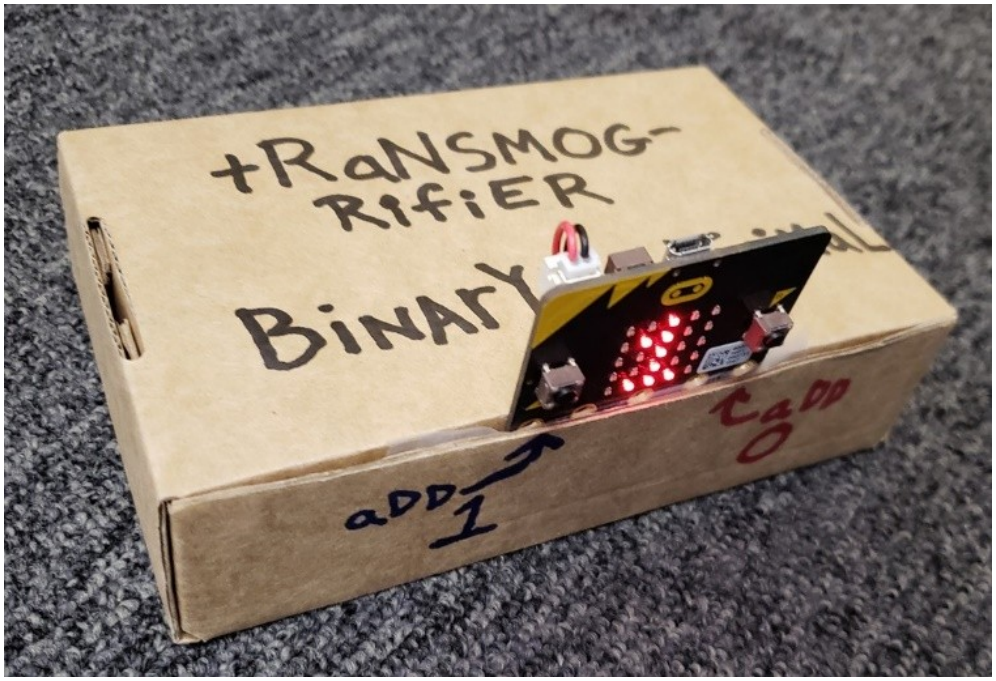
Try it out!

19. Once you’ve tested all the code in the Simulator, download it to the micro:bit. Have someone else try your program out. Then think about how the program might be improved.

Mod ideas

Here are some additional modifications you might try:

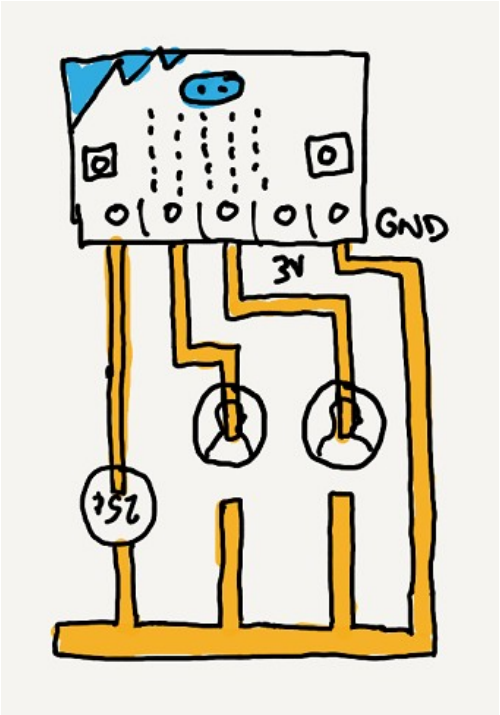
- Add a way to clear the binary and decimal values so you can start over.
- Add a way to erase the previous value.
- Create a decimal-binary converter that allows you enter a decimal value and see the binary equivalent when you press A+B.
- Create a physical housing or case for your binary calculator!



Lesson C: Make a binary cash register

Activity: Binary cash register project

The unplugged activity in Lesson A uses a vending machine as a model for creating different combinations of binary place values. You found that for *n* coins, there is one—and only one—way to make every number between 0 and 2^{n-1}.



For this project, you will invent a paper and cardboard version of the binary counter, then program it to display the decimal value of those numbers.

Project expectations

Follow the design thinking approach and make sure your project meets these specifications:

- Uses binary in a way that's integral to the program
- Uses mathematical operations to convert decimal to binary
- All binary numbers display correctly
- The program compiles and runs as intended and includes meaningful comments in code
- Provide the written Reflection Diary entry

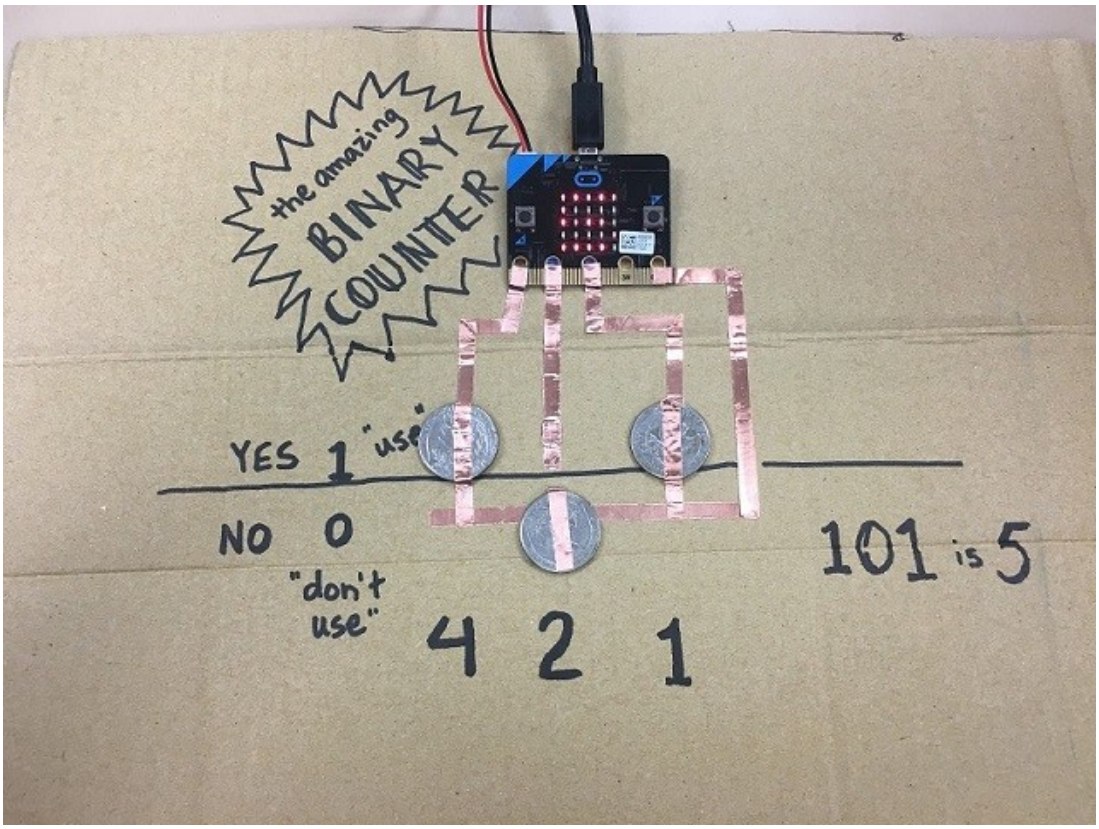
The design thinking process:

1. Empathize by learning more about your target audience.
2. Define: Understand and identify your audience's problems or needs.
3. Ideate: Brainstorm several possible creative solutions.
4. Prototype: Construct rough drafts or sketches of your ideas.
5. Test your prototype solutions.

Use this space to start your design thinking process:

Project example

Binary cash register

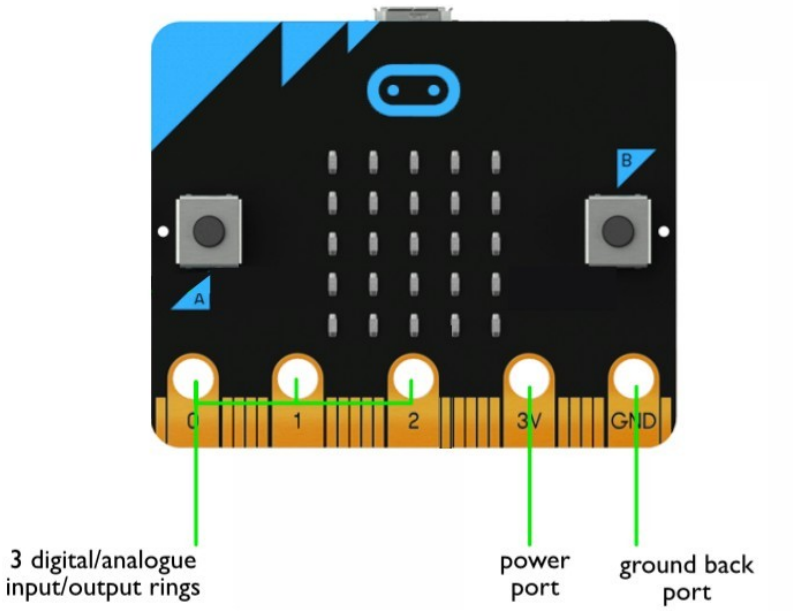


This is one possible design for a binary cash register. It uses coins and copper tape on a piece of cardboard. Normally, to indicate “off” or 0, the coins are flipped up. And to indicate “on” or 1, the coin is flipped so it lays flat across both pieces of copper tape, completing the circuit so the micro:bit can detect that that pin has been activated, and calculates and displays the decimal value of the binary number that is indicated by the coins.

Copper tape is a thin, flexible strip of copper with an adhesive back. Usually, copper tape can conduct electricity even through the sticky side, but if you are sticking one piece of copper tape to another, be sure to go over the connection with your fingernail, pressing it down firmly.

Because the micro:bit only has three pins/rings, this binary register is limited to three place values. You might use variables to represent each of the three place values, or they can simply keep a running total by adding the appropriate amount when each of the three pins is pressed.

You will need to connect the ground (GND) pin using copper tape to the other side of the circuit – using the coin to connect them.



You can stick the micro:bit into place using some sticky tape, or you can create an actual holder. The copper tape connections are delicate though, so be careful when plugging and unplugging the power cable from the board.

Project mod options

Mods for the binary cash register

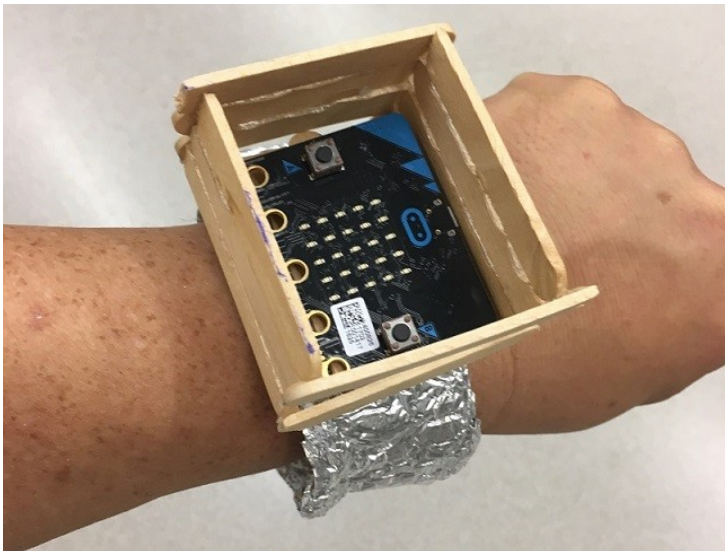
- Write some code that will display the number in binary when you press the A button.
- Think of a way to create more place values, perhaps by using a second micro:bit and a Radio connection.

Optional additional project: Build a binary wristwatch

Consider this optional project if you finish the binary cash register project early.

- Write a program that will display the correct time (once set) on the micro:bit.
- The three to four numbers displayed will be in binary (not decimal).
- To make the strap of the wristwatch, put two pieces of duct tape back-to-back, and use Velcro tabs as the fasteners.

This is another, more rigid holder design that allows the micro:bit to be worn on the wrist.



Project scoring rubric

Assessment elements	1	2	3	4
Binary display	No binary numerals display correctly.	At least one binary numeral displays.	At least two binary numerals display correctly.	All binary numerals display correctly.
micro:bit program	micro:bit program lacks three or more of the required elements.	micro:bit program lacks two of the required elements.	micro:bit program lacks one of the required elements.	micro:bit program: ▪ Uses binary in a way that is integral to the program ▪ Uses mathematical operations to convert decimal to binary ▪ Compiles and runs as intended ▪ Uses meaningful comments in code

Reflection Diary

Expectations

Write a reflection of about 150–300 words addressing the following points:

- Describe what the physical component of your micro:bit project was (e.g., an armband, a cardboard mount, a holder, etc.)
- How well did your prototype work? What were you happy with? What would you change?
- What was something that was surprising to you about the process of creating this project?
- Describe one way in which your project differed from the example that was given. How would you recognize it as your own?
- Publish your MakeCode program and include the link.

Diary entry scoring rubric

Assessment elements	1	2	3	4
Diary entry	Diary entry is missing three or more of the required elements.	Diary entry is missing two of the required elements.	Diary entry is missing one of the required elements.	Diary entry addresses all elements.

Glossary of key terms

Binary digit: A number with only two possible values: 0 or 1.

Bit: A binary digit with two possible values: 0 or 1.

Byte: A byte is a sequence of binary digits made up of eight bits. It has 256 possible values from 00000000 through 11111111.

Kilobyte (KB): 1,024 bytes or 2^{10} bytes

Megabyte (MB): 1,048, 576 bytes or 2^{20} bytes

Terabyte (TB): 1,099,511,627,776 bytes or 2^{40} bytes

Gigabyte (GB): 1,073,741,824 bytes or 2^{30} bytes